MedCommand

EMS Protocol Application

Comprehensive Design, Implementation & User Manual

Version 2.1 | 2025

Prepared for:

EMS Agency Administrators & Field Personnel

Table of Contents

1. Introduction & Overview 4
2. System Architecture & Technical Design 5
   2.1 Deployment Modes — Server vs. Standalone 5
   2.2 Technology Stack 6
   2.3 File Structure 6
   2.4 REST API Reference 7
3. Installation & Deployment 8
   3.1 Requirements 8
   3.2 Node.js Server Installation (Recommended) 8
   3.3 Running as a System Service (Linux) 9
   3.4 Reverse Proxy & HTTPS with Nginx 10
   3.5 Cloud & Hosting Platforms 11
   3.6 Standalone / Static HTML Mode 11
   3.7 Progressive Web App (PWA) Installation 12
4. Admin Configuration Guide 13
   4.1 Accessing the Admin Panel 13
   4.2 Server vs. Standalone Authentication 13
   4.3 Managing Protocol Sets 14
   4.4 Managing Category Tags 14
   4.5 Managing Skill Level Colors 15
   4.6 Creating & Editing Protocols 16
   4.7 Saving New Protocols — Method Comparison 17
   4.8 Changing the Admin Password 19
   4.9 Web-Based Protocol Editor (/admin) 20
5. Writing Protocols in CommonMark Markdown 22
   5.1 Skill-Level Section Fencing 22
   5.2 Markdown Quick Reference 23
   5.3 Linking Between Protocols 24
6. End-User Guide 25
   6.1 Home Screen & Server Status Badge 25
   6.2 Protocol Sets 25
   6.3 Browsing by Category & Skill Level 26
   6.4 Search 26
   6.5 Reading a Protocol & Skill-Level Sections 26
   6.6 Bookmarks 27
   6.7 Clinical Calculators 27
   6.8 Day & Night Mode 27
   6.9 Mobile Interface (iPhone & Android) 28
7. Clinical Calculators Reference 29
8. Offline Operation 30
9. Data Storage & Privacy 31
10. Updating the Application 32

11. Troubleshooting 33
12. Frequently Asked Questions 34
13. Appendix A: REST API Reference 35
14. Appendix B: Markdown Cheat Sheet 37

1. Introduction & Overview

MedCommand is a browser-based EMS Protocol Application designed for use by emergency medical services agencies of all sizes. It runs on laptops, tablets, and mobile phones — and works completely offline after initial load. Version 2.0 introduces a Node.js/Express backend server that enables real-time, multi-device synchronisation: protocols created or edited by an administrator are instantly available to every device connected to the server. The application can also run in standalone mode without a server, falling back to browser localStorage for single-device deployments.

Key Features at a Glance

Feature	Description
Server-Backed Storage	Node.js/Express REST API writes protocols to data.json; all devices see the same content instantly
Standalone / Offline Mode	Works without a server using browser localStorage; status badge shows current mode
Full-Text Search	Searches protocol titles, body content, and tags in real time across all sets
CommonMark Markdown	All protocol content written in standard Markdown with full formatting support
Skill-Level Fencing	Wrap any block of steps in ::: emt / ::: paramedic fences — renders as color-coded sections
Multiple Protocol Sets	Separate ALS, BLS, or specialty sets selectable from a single interface
Clinical Calculators	8 built-in calculators: GCS, APGAR, Parkland, Shock Index, Stroke Scale, and more
Secure Admin Auth	SHA-256 hashed password stored server-side; 4-hour expiring Bearer token session
Auto-Refresh on Launch	Last refresh date/time shown on Home Screen; server timestamp updated on every launch
Revision Dates	Each protocol displays its last revision date
Protocol Linking	Link from one protocol to another using [text](protocol:id) syntax
Category Tags	Admin-defined sections with custom colors; 9 pre-built categories
Skill Level Colors	Color-coded card borders and section headers; fully admin customizable
Day & Night Mode	Toggle dark/light themes; preference persists locally across sessions
Bookmarks	Users star protocols for quick access; bookmarks stored locally per device

Responsive Design	Optimized layout for phones, tablets, and desktops; PWA-installable
Offline Capable	Service Worker caches app shell; all reading works without internet after first load

2. System Architecture & Technical Design

2.1 Deployment Modes — Server vs. Standalone

Version 2.0 introduces two distinct deployment modes. The application detects which mode it is running in automatically on startup and displays a status badge in the top-right corner of the interface.

Aspect	Server Mode (Green badge)	Standalone Mode (Yellow badge)
How to run	npm start (Node.js server)	Open index.html directly in browser
Protocol storage	data.json on the server	Browser localStorage
Multi-device sync	Yes — all devices see same data	No — each device is independent
Admin auth	SHA-256 hashed, server-side token	Password stored in localStorage
Password security	Cryptographically hashed	Plain text (not recommended for production)
Data survives browser clear	Yes	No
Offline reading	Yes (Service Worker)	Yes (Service Worker)
Recommended for	All agency deployments	Single-device or testing use

2.2 Technology Stack

- Node.js / Express — REST API server; serves index.html and all /api/ routes
- data.json — flat-file JSON database; all protocol content, sets, tags, levels, and hashed password
- HTML5 / CSS3 / Vanilla JavaScript — zero frontend framework; no build step required
- marked.js (CDN) — CommonMark Markdown parser for protocol rendering
- Fetch API — client-side HTTP calls to the REST API with Bearer token authentication
- localStorage — stores bookmarks, theme preference, active set; also full fallback in standalone mode
- Service Worker — caches application shell for offline reading
- CSS Custom Properties — powers the day/night theme system and dynamic skill-level colors
- crypto (Node built-in) — SHA-256 password hashing with salt; no external auth library needed

2.3 File Structure

```
medcommand/
  server.js        Node.js/Express server + REST API
  index.html       MedCommand web application (served by server.js)
  data.json        All protocol data — auto-created on first run
  package.json     Node.js dependencies (express, cors)
  README.md        Quick-start guide
```

```
index.html internal structure:
  <style>          CSS variables, layout, components, skill-section styles
  <body>           HTML: topbar (with server badge), sidebar, content pages
  <script>
    API layer      fetch() wrappers: apiGet, apiPost, apiPut, apiDelete
    State layer    loadState() — tries /api/state first, falls back to localStorage
    Auth           checkAdminLogin() — calls /api/auth in server mode
    CRUD           saveProtocol, deleteProtocol, addTag, addLevel... (all async)
    Renderer       renderProtocolContent() — splits on ::: fences, renders per-chunk
    Calculators    CALCULATORS[] array with calc() functions
    Service Worker Inline SW for offline caching
```

2.4 REST API Overview

All write endpoints require a Bearer token obtained from POST /api/auth. Read endpoints are public — no token required. Tokens expire after 4 hours.

Method	Route	Auth	Description
GET	`/api/state`	No	Full application state (no password hash)
POST	`/api/auth`	No	Login — body: {password} → returns {token}
POST	`/api/refresh`	No	Update lastRefresh timestamp
GET	`/api/protocols`	No	List all protocols
POST	`/api/protocols`	Yes	Create a new protocol
PUT	`/api/protocols/:id`	Yes	Update an existing protocol
DELETE	`/api/protocols/:id`	Yes	Delete a protocol
GET	`/api/sets`	No	List protocol sets
POST	`/api/sets`	Yes	Create a protocol set
DELETE	`/api/sets/:id`	Yes	Delete a protocol set
GET	`/api/tags`	No	List category tags
POST	`/api/tags`	Yes	Create a tag
DELETE	`/api/tags/:id`	Yes	Delete a tag
GET	`/api/levels`	No	List skill levels
POST	`/api/levels`	Yes	Create a skill level
DELETE	`/api/levels/:id`	Yes	Delete a skill level
PUT	`/api/settings`	Yes	Change admin password

3. Installation & Deployment

3.1 Requirements

Component	Requirement
Node.js	Version 16 or higher — download from nodejs.org
npm	Included with Node.js — used to install express and cors
Operating System	Linux (recommended), Windows Server, or macOS
Browser (Client)	Chrome 80+, Firefox 75+, Safari 13+, Edge 80+, or any modern mobile browser
HTTPS	Required for Service Worker / PWA installation — use a reverse proxy for SSL
Disk Space	Approximately 10 MB for the application and node_modules

3.2 Node.js Server Installation (Recommended)

This is the standard deployment method for all agency installations. It enables real-time multi-device synchronisation.

```
# 1. Transfer the medcommand.zip file to your server, then unzip it
unzip medcommand.zip
cd medcommand

# 2. Install Node.js dependencies (one time only)
npm install

# 3. Start the server
npm start

# The server starts on port 3000 by default.
# Access at: http://your-server-ip:3000

# To run on a different port:
PORT=8080 npm start
```

First Run

On first start, the server creates data.json automatically with the default 9 tags, 3 skill levels, and 2 protocol sets. No protocols are pre-loaded — add them via the Admin Panel or embed them in index.html before deployment.

3.3 Running as a System Service (Linux)

For production deployments, run MedCommand as a systemd service so it starts automatically on boot and restarts on failure.

1. Create the service file:

```
sudo nano /etc/systemd/system/medcommand.service
```

2. Paste the following content (adjust paths as needed):

```
[Unit]
```

```
Description=MedCommand EMS Protocol Server
After=network.target

[Service]
Type=simple
User=www-data
WorkingDirectory=/opt/medcommand
ExecStart=/usr/bin/node server.js
Restart=on-failure
RestartSec=5
Environment=PORT=3000

[Install]
WantedBy=multi-user.target
```

3. Enable and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable medcommand
sudo systemctl start medcommand

# Check status
sudo systemctl status medcommand

# View logs
sudo journalctl -u medcommand -f
```

3.4 Reverse Proxy & HTTPS with Nginx

HTTPS is required for the Service Worker and PWA installation. The recommended approach is to place MedCommand behind Nginx with a free certificate from Let's Encrypt.

```
# 1. Install Nginx and Certbot
sudo apt update && sudo apt install nginx certbot python3-certbot-nginx -y

# 2. Create Nginx site config
sudo nano /etc/nginx/sites-available/medcommand
# Paste this configuration:
server {
    listen 80;
    server_name protocols.youragency.org;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
# 3. Enable site and obtain SSL certificate
sudo ln -s /etc/nginx/sites-available/medcommand /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx

# 4. Get free SSL certificate (replaces your domain)
sudo certbot --nginx -d protocols.youragency.org

# Certbot automatically configures HTTPS and sets up auto-renewal
# Access at: https://protocols.youragency.org
```

3.5 Cloud & Hosting Platforms

Render.com (Free Tier — Easiest)

4. Create a free account at render.com
5. Push the medcommand folder to a GitHub repository
6. New → Web Service → connect your GitHub repo
7. Build Command: npm install | Start Command: npm start
8. Render provides HTTPS automatically on a free subdomain
9. Note: on the free tier the server sleeps after 15 minutes of inactivity — use a paid tier for production

DigitalOcean / AWS EC2 / Linode

10. Create a Linux VPS (Ubuntu 22.04 recommended, 1 GB RAM minimum)
11. Follow Section 3.2 to install Node.js and the application
12. Follow Section 3.3 to set up a systemd service
13. Follow Section 3.4 to configure Nginx and HTTPS

3.6 Standalone / Static HTML Mode

The standalone HTML file (ems-protocol-app.html or index.html) can be opened directly in a browser without any server. This is suitable for single-device use, testing, or air-gapped environments.

- No installation required — simply open the file in a browser
- All data is stored in the browser's localStorage
- Changes are not shared with other devices
- The server status badge displays a yellow "Local" indicator
- Service Worker will not register over file:// protocol — for offline SW caching, use a simple local server:

```
# Python 3 (any platform)
cd /path/to/medcommand
python3 -m http.server 8080
# Access at http://localhost:8080
```

3.7 Progressive Web App (PWA) Installation

When served over HTTPS, MedCommand can be installed as a native-feeling app on any device.

iOS (iPhone / iPad)

14. Open Safari and navigate to the application URL
15. Tap the Share button (box with upward arrow)
16. Tap Add to Home Screen

17. Name it "EMS Protocols" and tap Add

Android

18. Open Chrome and navigate to the application URL
19. Tap the three-dot menu and select Add to Home Screen or Install App
20. Tap Install to confirm

Desktop (Chrome / Edge)

21. Navigate to the application URL
22. Click the install icon in the address bar or use the browser menu
23. Click Install — the app opens as a standalone window

4. Admin Configuration Guide

4.1 Accessing the Admin Panel

The Admin Panel is password-protected. The default password is: admin123

> **Security Recommendation**
> Change the default admin password immediately upon deployment. In server mode, the password is SHA-256 hashed on the server and never exposed to the client. In standalone mode, the password is stored in localStorage — use a strong password but be aware it offers limited cryptographic protection in standalone mode.

To access the Admin Panel:

24. Click the gear icon in the top-right corner of the application
25. The Admin Login dialog will appear
26. Enter the admin password and press Enter or click Login
27. The Admin Settings page opens

To exit the admin session, navigate away by clicking Home or any other section. The session token expires automatically after 4 hours in server mode and must be re-entered.

4.2 Server vs. Standalone Authentication

The authentication mechanism differs between modes:

Aspect	Server Mode	Standalone Mode
Password storage	SHA-256 hash in data.json (server-side)	Plain text in localStorage (client-side)
Login process	POST /api/auth → returns Bearer token	Compared directly in browser
Session duration	4 hours — then re-login required	Until page is closed or refreshed
Password reset	Edit data.json adminPasswordHash field, or use Admin Panel	Clear localStorage in browser DevTools
Security level	Production-suitable	Development / single-device only

4.3 Managing Protocol Sets

Protocol Sets are separate collections of protocols. A typical agency might have an ALS (Advanced Life Support) set and a BLS (Basic Life Support) set. Each set is independently selectable by end users from the sidebar dropdown.

Adding a Protocol Set

28. In the Admin Panel, locate the Protocol Sets section
29. Enter a Set Name (e.g., "ALS Protocols 2025") in the name field

30. Optionally enter a Refresh URL — this is a placeholder for future remote refresh functionality
31. Click Add Set
32. The new set appears in the set list and becomes available in the sidebar dropdown

Removing a Protocol Set

33. In the Protocol Sets section, click the ✕ button next to the set to remove
34. Note: You cannot remove the last remaining set
35. Protocols assigned to a deleted set remain in the database but will not be visible; reassign them first

> 📋 **Best Practice**
> Create your protocol sets before creating protocols. Each protocol must be assigned to a set. Use clear, year-stamped names (e.g., "County EMS ALS 2025") to make version management easy.

4.4 Managing Category Tags

Tags organize protocols into categories. Nine default categories are pre-configured: Medical, Trauma, Pediatric, Cardiology, Respiratory, Obstetrics, Procedure, Administrative, and Appendix. Administrators can add or remove tags at any time.

Adding a Tag

36. In the Admin Panel, locate the Category Tags section
37. Enter the Tag Name (e.g., "Toxicology")
38. Click the color picker to choose a display color for the tag
39. Click Add Tag
40. The tag immediately appears in the sidebar navigation and is available when creating protocols

Removing a Tag

41. Click the ✕ button next to the tag in the admin tag list
42. Note: Removing a tag does not remove protocols that used that tag — the tag reference will simply become an unresolved ID on those protocols

Tag Color Guidelines

- Use distinct, high-contrast colors for each tag to aid quick visual identification
- Consider color-coding by clinical domain: blues for medical, reds for trauma, greens for pediatric
- Avoid colors that are too similar to skill level colors to prevent confusion

4.5 Managing Skill Level Colors

Skill levels define the minimum certification level required to perform interventions in a protocol. Default levels are EMT (blue), AEMT (teal), and Paramedic (purple). Each level has a color that

appears as a left-side border accent on protocol cards, as a colored badge on protocol detail pages, and as the background of skill-level fence section headers within protocol bodies.

> **Automatic Text Contrast**
> MedCommand automatically calculates whether to display black or white text on skill-level section header backgrounds based on the chosen color's perceived luminance. Dark colors (blue, purple, teal, red) display white text. Light colors (yellow, amber, orange, light green, white) display black text. This means any color can be chosen freely from the color picker without risk of unreadable text.

Adding a Skill Level

43. In the Admin Panel, locate the Skill Level Colors section
44. Enter the Level Name (e.g., "EMT-B", "Critical Care Paramedic", "First Responder")
45. Select a color using the color picker — any color works; text contrast is handled automatically
46. Click Add Level
47. The new level is immediately available when creating or editing protocols and its fence ID is the lowercase version of the name (spaces replaced with underscores)

Removing a Skill Level

48. Click the ✕ button next to the level
49. Protocols previously assigned that level will retain the level ID but the section header will fall back to a neutral gray

Color Selection Guidelines

- Any color is safe — the app automatically picks black or white text for legibility
- Use sufficiently saturated colors so the section header banner is visually distinct from surrounding content
- Very pale or washed-out colors may be hard to distinguish as a left-border accent on protocol cards — prefer mid-to-dark tones for card borders
- Keep skill level colors visually distinct from category tag colors to avoid confusion in the sidebar

Default Color Scheme

Level	Default Color	Hex Code	Header Text
EMT	Blue	`#2979ff`	White
AEMT	Teal	`#00bcd4`	White
Paramedic	Purple	`#9c27b0`	White

4.6 Creating & Editing Protocols

All protocol content is written in CommonMark Markdown (see Section 5 for full syntax reference). Protocols can be created, edited, or deleted from the Admin Panel.

Creating a New Protocol

50. In the Admin Panel, click the + New Protocol button in the Manage Protocols section
51. The Protocol Editor modal opens with the following fields:
    - Title — the protocol name as it will appear in lists and search results
    - Protocol Set — which set this protocol belongs to
    - Tags — comma-separated tag IDs (e.g., cardiology, medical)
    - Skill Level — the minimum certification level required
    - Revision Date — the date this protocol was last clinically reviewed
    - Content — the protocol body written in CommonMark Markdown
52. Write or paste your Markdown content in the Content field
53. Click the Preview tab to see how the protocol will render
54. Click Save when done

Editing an Existing Protocol

55. In the Admin Panel, scroll to the Manage Protocols section
56. Click any protocol in the list to open the editor with that protocol's content pre-loaded
57. Make your changes
58. Click Save to update

Deleting a Protocol

59. Open the Protocol Editor for the protocol you want to delete
60. Click the Delete button (shown in red at the bottom of the editor)
61. Confirm deletion in the dialog
62. Note: Deletion is immediate and cannot be undone. Bookmarks referencing the deleted protocol are automatically cleaned up.

> 💡 **Tip: Bulk Content Migration**
> To add many protocols at once, consider editing the HTML source file directly. Find the loadDefaultProtocols() function in the script section and add your protocol objects to the array before deployment. This pre-populates all installations without requiring admin entry on each device.

4.7 Saving New Protocols — Method Comparison

There are two distinct ways to save protocols into MedCommand. Understanding the difference is essential for multi-device agency deployments.

Situation	Method 1 — Admin Panel	Method 2 — Embed in HTML
Testing / drafting a protocol	Recommended	Not needed
Single-device use only	Sufficient	Not needed
Publishing to all agency	This device only	Required

devices		
Multi-unit / multi-device deployment	This device only	Required
No code editing needed	Yes	Requires text editor
Survives browser data clear	Data may be lost	Always present

Method 1 — Admin Panel (saves to current device/browser)

This is the standard day-to-day method for creating and editing protocols. No code editing is required.

63. Click the gear icon in the top-right corner and log in with the admin password
64. In the Manage Protocols section, click + New Protocol
65. Complete all fields: Title, Protocol Set, Tags, Skill Level, Revision Date, and Content
66. Write protocol content using CommonMark Markdown with skill-level fence sections (see Section 5)
67. Click the Preview tab to verify formatting and skill-level section rendering
68. Click Save — the protocol is immediately stored in localStorage and visible on the Home Screen

Device Limitation

Protocols saved via the Admin Panel are stored in the browser localStorage on that specific device and browser only. They are not visible to other users on other devices. If the browser data is cleared, these protocols will be lost. For agency-wide deployment, use Method 2.

Method 2 — Embed in the HTML Source File (deploys to all devices)

This method permanently embeds protocols into the application file itself. Any device that loads the updated HTML file will have the protocols regardless of localStorage state. This is the required method for agency-wide distribution.

69. Open ems-protocol-app.html in a text editor such as Notepad++, VS Code, or Sublime Text
70. Use Find (Ctrl+F) to search for the text: loadDefaultProtocols
71. Locate the protocols array inside that function — it begins with: state.protocols = [
72. Add a new protocol object to the array before the closing]; following the template shown below
73. Save the file and redeploy it to your web server, replacing the previous version
74. All devices that load the new file will immediately have access to the new protocol

Protocol Object Template

```
{
  id: 'p009',               // Unique ID — increment for each new protocol
  setId: 'set1',            // 'set1' = ALS set, 'set2' = BLS set
  title: 'Hypoglycemia',
```

```
  tags: ['medical'],        // Must match tag IDs in Admin > Category Tags
  level: 'emt',             // Must match a level ID in Admin > Skill Level Colors
  revised: '2025-04-24',    // ISO date format: YYYY-MM-DD
  content: `## Hypoglycemia Protocol

**Indications:** Blood glucose < 60 mg/dL with symptomatic patient.

::: emt
### EMT Interventions
- Oral glucose 15g PO if patient conscious and able to swallow
- Monitor airway; reassess glucose in 15 minutes
:::

::: aemt
### AEMT Interventions
- All EMT interventions above
- IV/IO access; Dextrose 50% (D50) 25g IV if unable to take PO
:::

::: paramedic
### Paramedic Interventions
- All AEMT interventions above
- Glucagon 1mg IM/IN if no IV access obtainable
- Recheck glucose 10 minutes after treatment
:::
`
},
```

> 💡 **Tip: Draft in Admin Panel, then Embed**
>
> The recommended workflow is to draft and preview a new protocol using the Admin Panel (Method 1), then once it is clinically reviewed and approved, copy the Markdown content into the HTML source file (Method 2) for agency-wide distribution.

Important Notes on Protocol IDs

- Each protocol must have a unique id value — no two protocols can share the same ID
- Default protocol IDs are p001 through p008 — start new embedded protocols at p009 and increment
- Admin Panel-created protocols receive IDs automatically in the format p_ followed by a timestamp
- Protocol IDs are used for cross-reference links — e.g., [See Cardiac Arrest](protocol:p003)
- If you change an existing protocol ID, any links pointing to the old ID will break

4.8 Changing the Admin Password

In server mode, the password is securely hashed server-side using SHA-256. To change it:

75. Access the Admin Panel using the current password
76. Scroll to the Admin Password section at the bottom of the page
77. Type the new password in the input field
78. Click Update
79. In server mode, the new hashed password is written to data.json immediately
80. In standalone mode, the password is updated in localStorage

81. Test the new password by navigating to Home and logging back in

Forgot Your Password? (Server Mode)

Stop the server. Open data.json in a text editor. Delete the adminPasswordHash field entirely. Restart the server — it will regenerate with the default password admin123. Log in and set a new password immediately.

Password Reset Utility

The medcommand package includes reset-password.js for quick password recovery. Run: node reset-password.js (resets to admin123) or node reset-password.js newpassword (sets a specific password). The server can be running or stopped when this is executed.

4.9 Web-Based Protocol Editor (/admin)

Version 2.0 introduced a dedicated, full-featured web-based protocol editor accessible at http://your-server:3000/admin. This is the recommended interface for all protocol authoring when running in server mode. Changes made here are saved immediately to data.json and are live for all connected users instantly.

Accessing the Editor

82. Ensure the Node.js server is running (npm start)
83. Open a browser and navigate to http://your-server-ip:3000/admin
84. Enter the admin password at the login screen and press Enter or click Sign In
85. The session is stored in the browser tab's sessionStorage — closing the tab logs you out; refreshing keeps you logged in
86. Sessions expire after 4 hours and require re-login

Editor Layout

The editor is divided into three areas:

- Left sidebar — protocol list with set filter tabs, live search, and a + New Protocol button
- Center — write pane with the Markdown editor
- Right — live preview pane that renders the protocol exactly as end users see it, including skill-level fence sections
- Top toolbar — fields for Title, Protocol Set, Skill Level, Tags, and Revision Date; Save and Delete buttons

Creating a Protocol

87. Click + New Protocol in the sidebar
88. A template is pre-populated with fence sections for every currently defined skill level
89. Enter a title in the Title field at the top
90. Select the Protocol Set, Skill Level, and enter Tags (comma-separated)
91. Write protocol content in the left (Write) pane — the right (Preview) pane updates live
92. Press Ctrl+S (or Cmd+S on Mac) or click Save Protocol to save
93. The protocol is immediately written to data.json and visible to all users

Editing an Existing Protocol

94. Click any protocol in the left sidebar list to load it into the editor
95. Use the set filter tabs or the search box to find specific protocols
96. Make changes in the write pane — the status badge shows "● Unsaved" when there are unsaved changes
97. Save with Ctrl+S or the Save Protocol button
98. The save status badge shows: ● Unsaved → ⏳ Saving → ✓ Saved (or ✕ Save failed)

Syntax Bar

A row of quick-insert buttons appears between the toolbar and the editor. Clicking any button inserts the corresponding Markdown snippet at the cursor position:

- Formatting: bold, italic, headings (H2, H3), lists, blockquotes, code, dividers
- Structure: table template, callout note block, horizontal rule
- Links: external URL and internal protocol cross-reference
- Skill-level fences: one button per defined level (e.g., ::: emt, ::: paramedic) inserts a ready-made fence block

Bulk Import

99. Click the Import button in the topbar
100. Paste a JSON array of protocol objects into the text area
101. Click Import Protocols
102. Protocols with matching IDs are updated; new IDs are created fresh
103. A summary toast shows how many were created, updated, or failed

```
# Minimum valid import format:
[
  {
    "title": "Hypoglycemia",
    "setId": "set1",
    "level": "emt",
    "tags": ["medical"],
    "revised": "2025-04-25",
    "content": "## Hypoglycemia\n\n::: emt\n..."
  }
]
```

Settings Panel

Click the Settings button in the topbar to manage protocol sets, category tags, skill levels, and the admin password — all without leaving the editor. Changes take effect immediately and are saved to data.json.

Keyboard Shortcuts

Shortcut	Action
`Ctrl+S / Cmd+S`	Save current protocol immediately
`Tab`	Insert 2 spaces (indent) at cursor position
`Escape`	Close any open modal (settings, import)

Admin Editor vs. In-App Admin Panel

The /admin editor (this section) is the recommended interface for all protocol work in server mode — it provides a full split-pane editing experience with live preview. The in-app Admin Panel (accessed via the gear icon in the main viewer) also allows editing and is the only option in standalone mode, but uses a simpler modal editor without a split-pane preview.

5. Writing Protocols in CommonMark Markdown

All protocol content is written in CommonMark Markdown — a standardized, plain-text format that renders into formatted HTML. MedCommand extends standard Markdown with a skill-level fence syntax that wraps sections of a protocol in color-coded, labeled blocks corresponding to certification levels.

5.1 Skill-Level Section Fencing

Skill-level fences allow you to mark any block of steps as belonging to a specific certification level. The fence renders as a colored header bar followed by the section content. Multiple fences can appear in any order and can interleave freely throughout a protocol.

Syntax

```
::: levelid        ← Opening fence (levelid must match a level defined in Admin)
### Section Title
- Step one
- Step two
:::              ← Closing fence (three colons on its own line)

# Example with all three default levels:

::: emt
### EMT Interventions
- Oxygen via NRB if SpO2 < 94%
- Aspirin 324mg PO (chewable)
:::

::: aemt
### AEMT Interventions
- All EMT interventions above
- IV access; 12-lead ECG acquisition
:::

::: paramedic
### Paramedic Interventions
- All AEMT interventions above
- Nitroglycerin 0.4mg SL q5min x3 if SBP > 90
:::

# Fences can repeat and interleave in any order
::: emt
### Transport (All Levels)
- Notify receiving facility en route
:::
```

How Fences Render

Each fenced block displays a colored header banner (e.g., a blue "EMT Scope" bar) using the color defined for that level in Admin Settings. A skill-level legend strip appears at the top of the protocol showing all levels present. Use the Preview tab in the editor to see fences rendered live before saving.

Fence Rules

Opening fences use ::: followed by a space and the level ID. Closing fences are ::: alone on their own line. Both must be on their own line with no leading spaces. Content between fences is rendered as standard Markdown including headings, lists, and tables. If a fence is opened but never closed, the

remaining content is treated as part of that section.

5.2 Markdown Quick Reference

Markdown Syntax	Result
`# Heading Level 1`	Large section heading (H1)
`## Heading Level 2`	Medium subsection heading (H2)
`### Heading Level 3`	Small subsection heading (H3)
`**bold text**`	Bold text
`*italic text*`	Italic text
`- Item one\n- Item two`	Unordered (bullet) list
`1. First\n2. Second`	Ordered (numbered) list
`> Blockquote text`	Highlighted callout / note
`` `inline code` ``	Monospace code snippet
`---`	Horizontal rule / divider
`| Col1 | Col2 |\n|------|------|\n| A | B |`	Table with headers and rows
`[Link Text](https://url)`	External hyperlink
`[Protocol Link](protocol:p001)`	Internal protocol cross-reference

5.2 Linking Between Protocols

MedCommand supports internal cross-references between protocols using a special link syntax. When a user taps a protocol link, the app navigates directly to the referenced protocol without a page reload.

Internal Link Syntax

```
[Link Text](protocol:PROTOCOL_ID)

# Example:
See [Cardiac Arrest Protocol](protocol:p003) for CPR guidelines.

# The protocol ID is the internal ID assigned when the protocol was created.
# For default protocols, IDs are p001 through p008.
# For admin-created protocols, IDs begin with p_ followed by a timestamp.
```

To find a protocol's ID: open the Admin Panel, click the protocol in the Manage Protocols list, and note the internal ID (visible in the page URL or browser console if needed). A simpler approach is to use consistent, human-readable IDs by editing the source code before deployment.

5.3 Sample Protocol Structure

```
## Protocol Title
```

```
**Indications:** Brief description of when this protocol applies.

---

### Initial Assessment
- Ensure scene safety
- Obtain vital signs
- Establish IV access

### EMT Interventions
- Oxygen if SpO2 < 94%
- Position of comfort

### Paramedic Interventions
- 12-lead ECG
- Nitroglycerin 0.4mg SL if SBP > 90

### Reference Table
| Drug | Dose | Route |
|------|------|-------|
| Aspirin | 324mg | PO |
| Nitroglycerin | 0.4mg | SL |

> **⚠️ Note:** Always contact medical direction for unusual presentations.

### Related Protocols
- See [Cardiac Arrest](protocol:p003)
```

6. End-User Guide

This section describes how field personnel use MedCommand. No login or account is required for read-only access. Users can search protocols, view content, use calculators, and save bookmarks.

6.1 Home Screen & Server Status Badge

The Home Screen is the first thing users see when opening MedCommand. It displays:

- Agency name / protocol set name in large heading
- Last Refresh date and time — confirming protocol currency
- Total protocol count for the active set
- Protocol cards grouped by category, each showing title, skill level badge, and category tags
- Bookmark star (★) on each card for quick saving

The top-right navigation bar shows a server status badge that indicates the current operating mode:

- Green "Server" badge — connected to the Node.js server; all changes sync to all devices in real time
- Yellow "Local" badge — running in standalone mode; changes saved to this browser only

Tap or click any protocol card to open the full protocol.

6.2 Selecting a Protocol Set

If the agency has multiple protocol sets (e.g., ALS and BLS), a dropdown selector appears at the top of the left sidebar. To switch sets:

104. Locate the dropdown at the very top of the sidebar (just below the search bar on mobile)
105. Tap the dropdown and select the desired protocol set
106. The Home Screen refreshes to show only protocols in the selected set
107. The category counts in the sidebar update to reflect the active set

> 📱 **Mobile Tip**
> On phones and small tablets, the sidebar is hidden by default. Tap the ☰ (hamburger) menu icon in the top-left to open it and access the set selector and category filters.

6.3 Browsing by Category & Skill Level

The left sidebar contains two navigation sections:

- Categories — lists all tags (Medical, Trauma, Cardiology, etc.) with protocol counts. Tap any category to filter the Home Screen to show only protocols with that tag.

- Skill Levels — lists EMT, AEMT, Paramedic (and any custom levels). Tap a level to filter protocols to only that certification level.

Tap any active filter again to clear it and return to all protocols. Only one filter can be active at a time.

6.4 Search

The search bar at the top of the screen searches all protocol content in real time. It is available on all pages.

- Type any word, drug name, procedure, or phrase
- Search results appear instantly as you type, spanning all protocol sets
- Results show the protocol title, skill level, and a highlighted excerpt showing where the search term appears in the protocol body
- Tap any result to open the full protocol
- Keyboard shortcut: Ctrl+K (Windows/Linux) or Cmd+K (Mac) focuses the search bar instantly

6.5 Reading a Protocol & Skill-Level Sections

Tapping a protocol card opens the Protocol Detail view. This view contains:

- Protocol title in large heading
- Skill level badge (color-coded with automatic contrast text)
- Category tags
- Last Revised date
- Full formatted protocol body including headers, bullet points, tables, callouts, and links

If the protocol uses skill-level fence sections, a legend strip appears at the top of the body showing the levels present (e.g., EMT, AEMT, Paramedic) with their color swatches. Each fenced section is displayed as a color-coded block with a labeled header bar — the text color on the header is automatically black or white depending on which is more readable against the level's chosen color.

> **Protocol Links**
> When a protocol references another protocol (e.g., "See Cardiac Arrest Protocol"), these appear as clickable blue links. Tapping them opens the referenced protocol directly. Use the Back button to return to the previous view.

6.6 Bookmarks

Any protocol can be bookmarked for quick access. Bookmarks are saved per device in the browser's local storage.

Adding a Bookmark

108. From any protocol card on the Home Screen, tap the ★ star icon in the top-right corner of the card
109. The star turns yellow, confirming the bookmark was saved

110. A toast notification appears confirming the bookmark

Viewing Bookmarks

111. Tap the ⭐ icon in the top-right of the navigation bar, or click Bookmarks in the sidebar
112. All bookmarked protocols appear as cards, in the same format as the Home Screen
113. Tap any card to open the protocol

Removing a Bookmark

114. Tap the yellow ★ star on any bookmarked card
115. The star returns to gray and the bookmark is removed

6.7 Clinical Calculators

Tap the Calculators link in the sidebar to access the clinical calculator library. Eight calculators are available:

- Glasgow Coma Scale (GCS)
- Parkland Burn Formula
- APGAR Score
- Pediatric Weight Estimate (Broselow)
- $ETCO_2$ Interpretation
- IV Drip Rate Calculator
- Cincinnati Stroke Scale
- Shock Index

To use a calculator:

116. Tap the calculator card
117. Enter or select the required values in the form fields
118. Tap Calculate
119. The result appears with the calculated value, clinical interpretation, and any relevant notes
120. Tap Close or press Escape to dismiss

6.8 Day & Night Mode

MedCommand supports both a dark (night) mode and a light (day) mode. Dark mode is the default, optimized for low-light conditions common during night shifts.

- Tap the moon icon in the top-right navigation bar to toggle between modes
- The icon changes to a sun in light mode
- The theme preference is saved automatically and persists between sessions
- All protocol content, cards, and calculators adapt automatically to the selected theme

6.9 Mobile Interface (iPhone & Android)

MedCommand is fully optimized for use on iPhones, Android phones, and tablets. The interface adapts to screen size automatically.

Mobile Topbar Layout

On phones (screens 600px wide or narrower), the topbar splits into two rows to make full use of the available width:

- Row 1: hamburger menu (☰) | MedCommand logo | mode/theme/admin icon buttons | server status dot
- Row 2: full-width search bar spanning the entire screen

The server status indicator is condensed to a small colored dot on mobile (green = server connected, yellow = local mode) to save space. On tablets and desktops, the full badge text is shown.

Navigation on Mobile

- Tap the ☰ hamburger icon in the top-left to open the sidebar
- The sidebar slides in from the left and can be dismissed by tapping outside it or pressing Escape
- All protocol cards, calculators, and detail views are single-column on phones

Installing as a Home Screen App

For the best mobile experience, install MedCommand as a Progressive Web App (PWA) on the device home screen. This provides a full-screen experience without browser chrome, faster loading, and offline access. See Section 3.7 for installation instructions on iOS and Android.

7. Clinical Calculators Reference

The following table provides a detailed reference for all eight built-in calculators.

Calculator	Inputs	Output	Clinical Use
GCS	Eye, verbal, motor response	Score /15 + severity classification	Neurological assessment, airway decision support
Parkland Burn	Weight (kg), TBSA burned (%)	Total 24h fluid (mL), 8h rate (mL/hr)	IV resuscitation for burn patients
APGAR	5 signs at 1 and 5 min	Score /10 + intervention guidance	Neonatal assessment immediately after delivery
Peds Weight	Age (years), formula selection	Estimated weight (kg)	Drug dosing when actual weight unavailable
$ETCO_2$	$ETCO_2$ value, clinical context	Interpretation + action guidance	CPR quality, ventilation monitoring, ROSC
IV Drip Rate	Volume, time, drop factor	Drops/min and mL/hr	Macro/micro-drip IV administration
Cincinnati Stroke	3 neurological signs	Score /3 + stroke alert decision	Prehospital stroke identification
Shock Index	Heart rate, systolic BP	Shock index value + severity	Early shock detection and severity grading

8. Offline Operation

MedCommand is designed for use in environments where internet connectivity may be unreliable — including rural responses, building interiors, and mobile command vehicles. All reading and reference functionality is available offline in both server and standalone modes.

Offline Behaviour by Mode

Feature	Server Mode (offline)	Standalone Mode
Browse protocols	Yes (cached by SW)	Yes (localStorage)
Full-text search	Yes	Yes
Clinical calculators	Yes	Yes
Bookmarks	Yes (localStorage)	Yes (localStorage)
Day/Night mode	Yes	Yes
Create/edit protocols	No (needs server)	Yes (saves locally)
Admin Panel	No (needs server auth)	Yes

How Offline Caching Works (Server Mode)

121. On first load over HTTPS, the Service Worker installs and caches the application shell (index.html and marked.js)
122. The app shell is served from the SW cache on all subsequent loads, even with no internet
123. Protocol data is loaded from the server on each launch when online; if the server is unreachable, the app falls back to the last locally-cached state and shows the yellow "Local" badge
124. The "Last Refresh" timestamp on the Home Screen reflects the last successful server contact

What Always Works Offline

- Reading and browsing all previously-loaded protocols
- Full-text search across loaded protocol content
- All 8 clinical calculators
- Bookmarks (stored locally)
- Day/Night mode toggle

Field Tip

Load MedCommand while you still have connectivity (e.g., at the station or dispatch) to ensure the Service Worker caches the app shell and protocols are loaded into memory. After that, the app works fully for protocol reference even in dead zones. Protocol edits require server connectivity.

9. Data Storage & Privacy

MedCommand v2.0 stores data in two locations depending on the deployment mode. No data is ever transmitted to any third-party server.

Server Mode — data.json

When running with the Node.js server, all protocol content, sets, tags, skill levels, and the hashed admin password are stored in data.json on the server. This file is the single source of truth for all connected devices.

- Location: medcommand/data.json on the server filesystem
- Format: plain JSON, human-readable and editable with any text editor
- Typical size: 50 KB – 2 MB for a full agency protocol library
- Backup: copy data.json regularly; this is the only backup needed for a full data recovery
- Admin password: stored as SHA-256(password + salt) — never the plain text password

Browser (localStorage) — Both Modes

The browser stores user-specific preferences and bookmarks locally regardless of mode:

- Bookmarks — which protocols the user has starred
- Theme preference — dark or light mode
- Active protocol set — which set was last selected

In standalone mode, localStorage also stores all protocol content, sets, tags, levels, and the admin password.

Data Loss Scenarios

Scenario	Impact & Mitigation
User clears browser data	Bookmarks and theme lost. Protocols unaffected (server mode). Full data lost (standalone).
Server disk failure	All protocol data lost if data.json not backed up. Back up data.json daily.
data.json deleted	Server recreates a blank default on next start. Restore from backup.
Private/incognito session	Bookmarks and theme not saved after tab closes. Protocols still readable from server.
User switches devices	Bookmarks do not transfer (they are per-device). Protocol content is always available from server.

Backup Procedure (Server Mode)

```
# Manual backup
cp /opt/medcommand/data.json /backups/data_$(date +%Y%m%d).json
```

```
# Automated daily backup via cron
crontab -e
# Add this line:
0 2 * * * cp /opt/medcommand/data.json /backups/medcmd_$(date +\%Y\%m\%d).json
```

HIPAA Consideration

MedCommand stores protocol content only — no patient data. It is not a patient care reporting (PCR) system and is not intended to store any protected health information (PHI). Agencies should ensure that users understand the app is for protocol reference only.

10. Updating the Application

MedCommand stores all protocol data in data.json (server mode) or browser localStorage (standalone mode), completely separate from the application code files. This means you can update the application without any risk of losing protocols.

Server Mode Update (Recommended)

Protocol data lives in data.json and is never touched by an update. The update process simply replaces the code files.

```
# 1. Back up your data first — always
cp data.json data.json.backup

# 2. Stop the server
sudo systemctl stop medcommand
# or press Ctrl+C if running in terminal

# 3. Unzip the new version
unzip medcommand.zip -d medcommand_new

# 4. Replace only the code files — never data.json
cp medcommand_new/medcommand/server.js    ./server.js
cp medcommand_new/medcommand/index.html   ./index.html
cp medcommand_new/medcommand/admin.html   ./admin.html
cp medcommand_new/medcommand/package.json ./package.json

# 5. Install any new dependencies
npm install

# 6. Restart the server
sudo systemctl start medcommand

# 7. Verify your data is intact
curl http://localhost:3000/api/protocols | python3 -m json.tool | head -20
```

Safe Rule

Never copy data.json from the new package over your existing data.json. The new package does not include a data.json — it is auto-created on first run if missing. Your existing data.json is always preserved as long as you do not explicitly delete or overwrite it.

Standalone HTML Mode Update

In standalone mode, protocols are stored in browser localStorage — completely independent of the HTML file. Replacing the HTML file at the same server path or file path does not affect localStorage.

- Static web server (Apache/Nginx): simply copy the new index.html to the web root — localStorage is unaffected
- Local file (same path): replace the file at the same location — localStorage survives
- Local file (new path or new device): export your data first, then import after opening the new file

Exporting Protocols Before Any Update

As a safety precaution before any update, export a copy of your protocols:

From the browser console (server mode)

```
// Open browser DevTools (F12), go to Console, paste:
copy(JSON.stringify(await fetch("/api/protocols").then(r=>r.json()), null, 2))
// Your clipboard now contains all protocols as JSON
// Save it to a .json file
```

From the browser console (standalone mode)

```
copy(localStorage.getItem("medcmd_local"))
// Paste into a text file and save
```

Restoring from export

If anything goes wrong during an update, use the Import button at /admin to re-import the exported JSON array. For standalone mode, run localStorage.setItem("medcmd_local", "PASTE_JSON_HERE") in the console and refresh.

11. Troubleshooting

Problem	Solution
Yellow "Local" badge when server should be running	Verify the Node.js server is running (npm start). Check that the browser is accessing the correct port. Check server logs: sudo journalctl -u medcommand -f
Cannot save protocol — "Save failed" error	Admin session may have expired (4-hour limit in server mode). Log out of Admin Panel and log back in to get a fresh token. In standalone mode, check that localStorage is not full.
Admin password not working	Default is admin123. In server mode, run: node reset-password.js to reset. Or open data.json, delete the adminPasswordHash line, and restart the server. In standalone mode, clear localStorage in browser DevTools → Application → Local Storage.
Server starts but app shows blank page	Verify index.html is in the same folder as server.js. Check that the file is named exactly index.html (not ems-protocol-app.html).
Protocols not showing on Home Screen	Ensure at least one protocol set exists and protocols are assigned to it. In server mode, check data.json has a protocols array with entries. In standalone mode, check localStorage in DevTools.
Skill-level section header text is hard to read	This was resolved in v2.1 — the app now automatically calculates whether black or white text gives better contrast against the chosen level color. Update to the latest version to get this fix.
Markdown not rendering (showing raw text)	The marked.js library failed to load from CDN. Check internet connectivity on first load, or embed marked.js inline in index.html for air-gapped deployments.
Skill-level fence sections not rendering	Verify opening fence is ::: followed by a space and the level ID (e.g., ::: emt). Verify closing fence is ::: alone on its own line. Ensure the level ID exactly matches one defined in Admin > Skill Level Colors (case-insensitive).
Bookmarks disappeared	Bookmarks are stored in browser localStorage. They are lost if browser data was cleared or an incognito session was closed. Protocol content is unaffected in server mode.
App not working offline	Verify the app is served over HTTPS — Service Workers require HTTPS. Check DevTools → Application → Service Workers for registration status.
Mobile topbar overflows or requires scrolling	Resolved in v2.1 — the topbar now uses a two-row layout on phones: row 1 has menu/logo/icons, row 2 has a full-width search bar. Update to the latest version.
Protocol links not working	Verify the protocol ID in the link matches an existing protocol ID exactly. IDs are case-sensitive. The Admin Panel shows all protocol IDs.
Sidebar not visible on mobile	Tap the three-line hamburger icon in the top-left corner to open

	the sidebar.
Port 3000 already in use	Run PORT=8080 npm start to use a different port, or find the conflicting process: lsof -i :3000 (Mac/Linux) or netstat -ano | findstr :3000 (Windows).

12. Frequently Asked Questions

Can multiple users share the same protocol data?

Yes — this is the primary purpose of server mode. Any device that connects to the server URL sees the same up-to-date protocols. Changes made by an admin are immediately visible to all connected users.

What is the /admin page and how is it different from the in-app Admin Panel?

The /admin page (http://your-server:3000/admin) is a dedicated web-based protocol editor with a split-pane Markdown editor and live preview, syntax bar, bulk import, and settings panel. The in-app Admin Panel (gear icon in the viewer) offers the same core functions in a modal dialog. The /admin editor is recommended for regular protocol authoring in server mode.

Can I still use the app without a server?

Yes. Open index.html directly in a browser — all features work using localStorage. The yellow "Local" badge indicates standalone mode. Changes are saved to that browser only and are not shared across devices.

Will I lose my protocols when I update the application?

No — not if you follow the correct procedure. In server mode, protocols live in data.json which is never touched by an update. Replace only the code files. In standalone mode, localStorage is independent of the HTML file. See Section 10 for full update instructions.

The skill-level section header text was unreadable — is this fixed?

Yes, fixed in v2.1. The app now uses the WCAG perceived luminance formula to automatically choose black or white text for each section header based on the background color. Any color from the picker is safe.

The topbar was too wide on iPhone — is this fixed?

Yes, fixed in v2.1. On screens up to 600px wide, the topbar uses a two-row layout: row 1 has the menu/logo/icons, row 2 has a full-width search bar. The server badge condenses to a colored dot on mobile.

Can I add images to protocols?

Yes — use Markdown image syntax: ![Alt text](https://url-to-image.jpg). Images must be hosted at an accessible URL. For offline images, embed them as base64 data URIs in the Markdown.

Is there a limit to how many protocols I can add?

In server mode, effectively no. In standalone mode, localStorage supports 5–10 MB per origin — sufficient for several hundred full-text protocols.

How do I back up all protocol data?

In server mode, copy data.json from the server. In standalone mode, open DevTools → Application → Local Storage, find medcmd_local, and copy the value. The /admin editor also supports bulk export via the browser console.

What happens if the server goes down while users are in the field?

All reading works — the Service Worker serves the cached app and protocols remain in memory. The yellow "Local" badge appears. Protocol editing requires server connectivity to be restored.

Can I integrate MedCommand with our CAD or ePCR system?

Not out of the box, but the REST API makes integration possible. See Section 13 (Appendix A) for the full API reference.

Can I add more clinical calculators?

Yes. Calculators are defined in the CALCULATORS array in index.html. Each has a name, description, fields array, and a calc() function. Follow the existing pattern — no server changes needed.

How do I migrate from standalone to server mode?

Install and start the server. Export protocols from the standalone browser console (JSON.parse(localStorage.getItem("medcmd_local"))), then use the /admin editor's Import button to bulk-import them into the server.

13. Appendix A: REST API Reference

This appendix provides a complete reference for the MedCommand REST API, including example request/response bodies. All endpoints are served by the Node.js server at the base URL http://your-server:3000.

Authentication

Obtain a token by posting the admin password to /api/auth. Include the token as a Bearer token in the Authorization header for all write operations. Tokens expire after 4 hours.

```
# Login
POST /api/auth
Content-Type: application/json

{"password": "admin123"}

# Response
{"token": "f4e4a06e9b2c94494cdd6007846743fb..."}

# Use token in subsequent requests:
Authorization: Bearer f4e4a06e9b2c94494cdd6007846743fb...
```

Protocol Operations

```
# Get all protocols (no auth required)
GET /api/protocols

# Create a new protocol
POST /api/protocols
Authorization: Bearer <token>
Content-Type: application/json

{
  "title": "Hypoglycemia",
  "setId": "set1",
  "tags": ["medical"],
  "level": "emt",
  "revised": "2025-04-25",
  "content": "## Hypoglycemia\n\n::: emt\n### EMT..."
}

# Response: the created protocol with its assigned ID
{"id": "p_1745123456789_ax3f2", "title": "Hypoglycemia", ...}

# Update a protocol
PUT /api/protocols/p_1745123456789_ax3f2
Authorization: Bearer <token>
{"title": "Hypoglycemia — Updated", ...}

# Delete a protocol
DELETE /api/protocols/p_1745123456789_ax3f2
Authorization: Bearer <token>
# Response: {"ok": true}
```

Tag, Set, and Level Operations

```
# All follow the same pattern: GET (public) / POST / DELETE (auth required)
```

```
GET    /api/tags
POST   /api/tags         {"id":"toxicology","name":"Toxicology","color":"#ff5722"}
DELETE /api/tags/toxicology

GET    /api/sets
POST   /api/sets         {"id":"set3","name":"Critical Care 2025","url":""}
DELETE /api/sets/set3

GET    /api/levels
POST   /api/levels       {"id":"cctp","name":"CCTP","color":"#00695c"}
DELETE /api/levels/cctp
```

Settings & Utility

```
# Change admin password
PUT /api/settings
Authorization: Bearer <token>
{"newPassword": "newSecurePassword99!"}
# Response: {"ok": true}

# Update lastRefresh timestamp
POST /api/refresh
# Response: {"lastRefresh": "2025-04-25T14:30:00.000Z"}

# Get full application state (no auth required)
# Returns sets, tags, levels, protocols, lastRefresh
# Never returns adminPasswordHash
GET /api/state
```

14. Appendix B: Complete Markdown Cheat Sheet

This appendix provides a comprehensive quick-reference for writing EMS protocols in CommonMark Markdown.

Text Formatting

```
**Bold text**                    → Bold
*Italic text*                    → Italic
***Bold and italic***            → Bold and italic
~~Strikethrough~~                → Strikethrough
`inline code`                    → Monospace code
```

Headings

```
# H1 — Protocol Title (largest)
## H2 — Major Section (e.g., "Initial Assessment")
### H3 — Subsection (e.g., "Paramedic Interventions")
#### H4 — Minor heading
```

Lists

```
- Unordered item one
- Unordered item two
  - Nested sub-item

1. Ordered step one
2. Ordered step two
   1. Nested ordered sub-step
```

Tables

```
| Header 1 | Header 2 | Header 3 |
|----------|----------|----------|
| Cell A   | Cell B   | Cell C   |
| Cell D   | Cell E   | Cell F   |

# Alignment:
| Left | Center | Right |
|:-----|:------:|------:|
| L    |   C    |     R |
```

Callouts & Emphasis Blocks

```
> Single-line blockquote / callout

> **⚠️ Warning:** Multi-line callout.
> Continue on next line.

> **💊 Drug Note:** Use for important medication callouts.

> **📋 Protocol Note:** Administrative or process notes.
```

Horizontal Rules & Line Breaks

```
---   ← Horizontal rule (creates a visual divider)

      ← Blank line between paragraphs (standard separation)

End a line with two spaces
to force a line break within a paragraph.
```

Links

```
[Link Text](https://www.example.com)    ← External link

[Protocol Name](protocol:p003)          ← Internal protocol link

# Protocol IDs:
# Default:   p001 through p008
# Custom:    p_ followed by creation timestamp (e.g., p_1712345678901)
```

Complete Protocol Template

```
## Protocol Name

**Indications:** When this protocol applies.

**Contraindications:** When NOT to use this protocol.

---

### Scene Size-Up
- Scene safety / BSI
- MOI / NOI

### Primary Assessment
- Airway, Breathing, Circulation, Disability, Exposure

### EMT Interventions
- List EMT-level actions here

### AEMT Interventions
- List AEMT-level actions here

### Paramedic Interventions
- List Paramedic-level actions here

### Medications
| Drug | Dose | Route | Frequency |
|------|------|-------|-----------|
| Drug Name | Xmg | IV/IO/IM | Once/q5min |

### Special Considerations
> **⚠️ Note:** Any warnings or important notes.

---

Revised: YYYY-MM-DD | Approved by: Medical Director

### Related Protocols
- [Related Protocol Name](protocol:pXXX)
```